

Understanding Self-Sovereign Identity

Concepts, Architecture

Table of Contents

1.	INTRODUCTION	2
2.	CORE SSI BUILDING BLOCKS:	2
3.	SSI TRUST TRIANGLE: HOW IT ALL WORKS TOGETHER.....	4
4.	VERIFICATION MODEL.....	9
5.	REFERENCE ARCHITECTURE (DID:ETHR EXAMPLE).....	10
6.	SECURITY & OPERATIONAL HIGHLIGHTS	11
7.	GETTING STARTED	12
8.	CONCLUSION	13
9.	GLOSSARY (ESSENTIAL TERMS)	14
10.	KEY REFERENCES.....	14

1. Introduction

Traditional digital trust systems rely on centralized providers or vulnerable document sharing, creating friction, privacy risks, and vendor lock-in. Most flows depend on:

1. Centralized identity providers (online verification for every transaction), or
2. Document sharing (PDFs/scans that are hard to validate and easy to over-collect)

This creates network/vendor dependence, privacy sacrifices (oversharing entire documents for a single fact), weak auditability, and manual fraud detection.

Self-Sovereign Identity (SSI) represents a paradigm shift—enabling entities (people, organizations, devices) to hold cryptographically verifiable credentials and present them directly to verifiers, without depending on a central authority for every transaction.

2. Core SSI Building Blocks:

○ Decentralized Identifiers (DIDs)

A DID is like a **digital passport number that YOU own and control** - not a company, not a government, but you.

- **What it is:** A globally unique identifier (like `did:ethr:0x123...`) that points to your identity
- **What makes it special:** Unlike email addresses or social media handles owned by companies (Google, Facebook), a DID is controlled entirely by you through cryptographic keys
- **How it works:** When you create a DID, you generate a key pair:
 - **Private Key** → Stays in your wallet (like a password, NEVER shared)
 - **Public Key** → Goes into your DID Document (shared publicly so others can verify your signatures)
- **DID Document:** A file containing your public key and other info, stored on a blockchain or registry. Anyone can look it up to verify your identity.

Think of it like this: Your DID is your “address” and your DID Document is your “profile” that others can look up - but only YOU have the keys to prove you own it.

○ Verifiable Credentials (VCs)

A VC is a **tamper-proof digital certificate** issued by a trusted authority.

- **What it is:** A digitally signed claim about you (e.g., “This person has a degree from MIT”)
- **Who creates it:** An issuer (university, employer, government) signs it with their private key
- **Why it’s trustworthy:** The signature can be verified using the issuer’s public key (from their DID Document)
- **Examples:** Digital diploma, driver’s license, employment proof, age verification

Think of it like a digital diploma that’s impossible to fake - anyone can verify it’s authentic without calling the university.

○ **Verifiable Presentations (VPs)**

A VP is a **package of credentials you choose to share** for a specific purpose.

- **What it is:** A wrapper containing one or more VCs, signed by you (the holder)
- **Why it exists:** You control what to share - show only what’s needed, nothing more
- **Selective disclosure:** Prove you’re over 21 without revealing your exact birth date or address

Think of it like showing a bouncer only the “Over 21” part of your ID, not your full name and address.

○ **Roles**

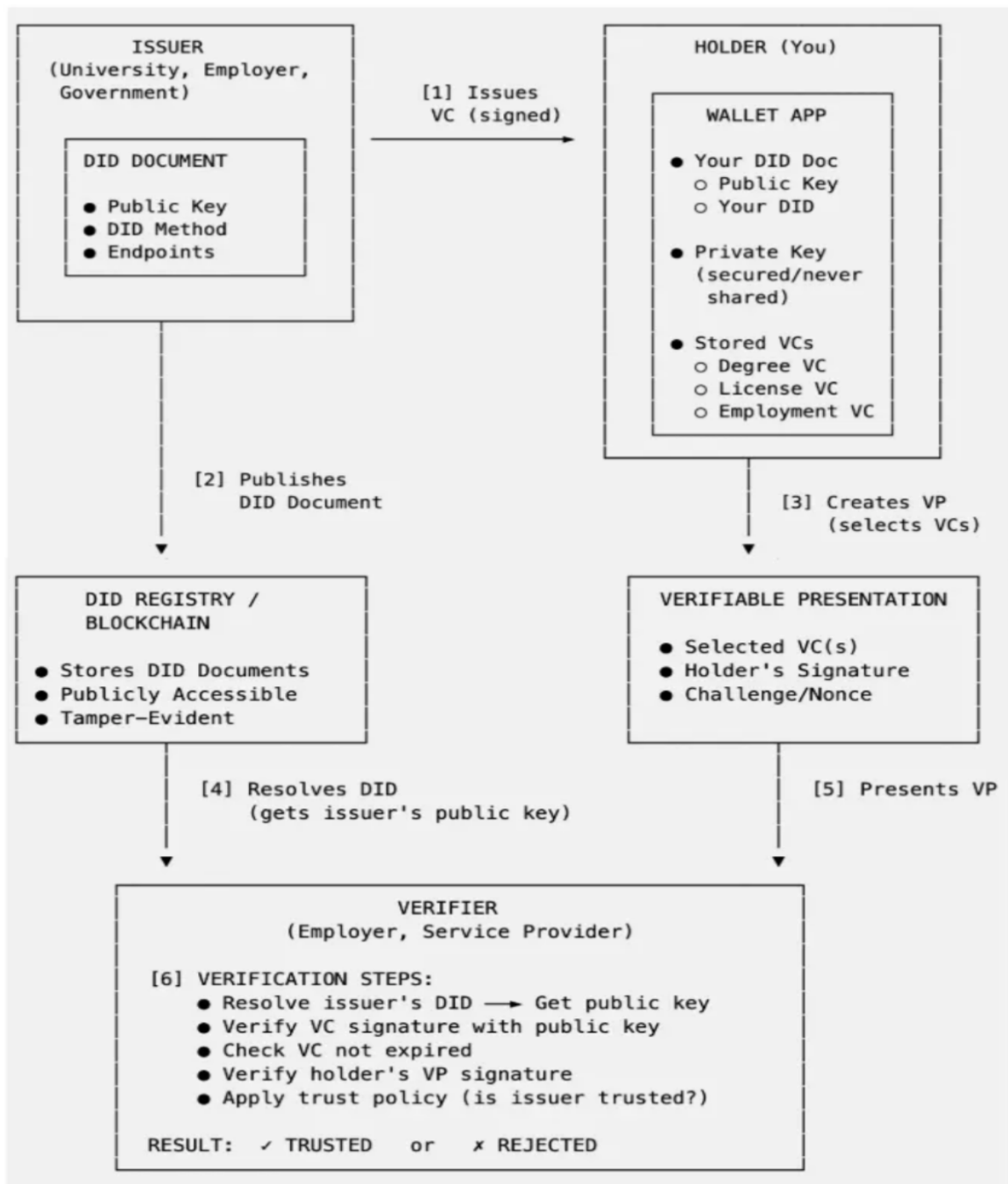
SSI introduces a clear separation of responsibilities among three parties. This role separation is fundamental: it decouples credential issuance from verification and puts holders in control of what they share.

The following table lists the three distinct roles in SSI

- **Issuer:** Asserts claims about a subject and issues cryptographically signed VCs (e.g., university issuing a degree credential).
- **Holder:** Receives and stores VCs; creates VPs that disclose only what’s necessary (e.g., candidate presenting a degree to an employer).
- **Verifier:** Requests proofs and validates them cryptographically + policy (e.g., employer verifying the degree VC).

Key insight: Cryptography answers “who signed this?”; policy answers “should we trust this issuer for this claim?”

3. SSI Trust Triangle: How It All Works Together



SSI Trust Triangle

- **The Flow (Step by Step)**

Step	Action	Description
0	Create DID	Generate key pair: Private Key (stored securely in wallet, NEVER shared) + Public Key . Create DID Document containing the public key. Register DID on blockchain (transaction signed by private key).
1	Issue VC	ISSUER creates a Verifiable Credential and signs it with issuer's private key (this signature proves the issuer created it)
2	Publish DID	ISSUER's DID Document (with public key) is available on blockchain/registry for anyone to resolve
3	Create VP	HOLDER selects which VCs to share and signs the VP with holder's private key (proves holder controls the VCs)
4	Resolve DID	VERIFIER looks up issuer's DID Document to get their public key
5	Present VP	HOLDER sends the VP to VERIFIER
6	Verify	VERIFIER uses issuer's public key to verify the VC signature

KEY INSIGHT: Verifier NEVER needs to contact the Issuer directly! The DID Document provides everything needed for verification.

SECURITY NOTE: The **Private Key never leaves the wallet**. It is used only to create signatures. Only the Public Key is shared (via DID Document), allowing others to verify signatures without accessing the private key.

WHAT GETS SIGNED? - DID Registration: Blockchain transaction is signed (for did:ethr) - proves ownership - **Verifiable Credential (VC):** Signed by ISSUER's private key - proves issuer created it - **Verifiable Presentation (VP):** Signed by HOLDER's private key - proves holder controls the VCs

○ **The Three Building Blocks**

	DID	VC	VP
What	Your Digital ID	Your Credential	Your Proof
Analogy	Like a unique username YOU control (not a company)	Like a digital diploma that can't be forged	Like showing ID but choosing what to reveal
Contains	• Your public key	• Claims about you	• Selected VCs
	• How to verify you	• Issuer's signature	• Your signature
		• Expiration date	• Proof of ownership

○ **How SSI Provides Trust, Security & Privacy**

TRUST	SECURITY	PRIVACY
✓ Credentials signed by trusted issuers	✓ Digital signatures prevent tampering	✓ YOU control your data
✓ Verifiers can check signatures without calling issuer	✓ Keys are never shared	✓ Share only what's needed (selective disclosure)
✓ No middleman needed for verification	✓ Credentials can't be copied or forged	✓ No central database tracking you
	✓ Blockchain anchoring provides extra proof	✓ Prove facts without revealing documents

○ **Real-World Example**

Scenario: Job Application - Proving your university degree to an employer

Traditional Way	SSI Way
Process:	Process:
1. Request official transcript from university	1. Open your wallet app
2. Wait days/weeks for delivery	2. Select your Degree VC

Traditional Way	SSI Way
3. Pay transcript fees	3. Create VP with only required info
4. Email/mail physical documents	4. Share instantly with employer
5. Employer calls university to verify	5. Employer verifies cryptographically
What You Share:	What You Share:
• Full transcript (all grades)	• Only: Degree type, Major, Graduation year
• Student ID number	• Verified by university's signature
• Date of birth	
• Full address history	What Stays Private:
• Social security number (sometimes)	• Your grades/GPA
	• Student ID number
Problems:	• Courses taken
• Slow (days to weeks)	• Date of birth
• Expensive (fees per transcript)	• Address
• Over-sharing sensitive data	
• Easy to forge paper documents	Benefits:
• Employer must trust the paper	• Instant verification
• No way to verify authenticity easily	• Free (no transcript fees)
	• Tamper-proof (cryptographically signed)
	• Privacy-preserving (share only what's needed)
	• Employer verifies without calling university

○ **Summary for Non-Technical Readers:**

Concept	Simple Explanation	Real-World Analogy
DID	Your unique digital identity that you control	Like having your own website address, but for identity
VC	A digital certificate signed by someone trusted	Like a diploma, but impossible to fake
VP	A way to prove something by showing only what's needed	Like showing a bouncer you're over 21 without revealing your address

The Key Innovation: In traditional systems, you hand over entire documents (passport, license, diploma). With SSI, you present *proofs* of specific facts, verified cryptographically, without oversharing personal information.

This paper presents a method-agnostic SSI model and a reference architecture approach with optional **SPV-like verification** for on-chain anchoring.

○ **The Problem with Current Digital Trust Systems**

Most digital trust flows rely on: 1. **Centralized identity providers** (online verification for every transaction), or 2. **Document sharing** (PDFs/scans that are hard to validate and easy to over-collect).

○ **Key Challenges:**

- **Network/vendor dependence:** availability, lock-in, integration complexity
- **Privacy sacrifice:** oversharing entire documents for a single fact
- **Weak auditability:** hard to explain “why we accepted this credential”
- **Manual fraud detection:** expensive, slow, error-prone

What’s missing: - Non-repudiation and integrity - Data minimization (prove facts without unnecessary disclosure) - Interoperability (portable proofs across systems) - Auditable verification decisions

○ **How SSI Solves These Problems:**

- **Integrity and authenticity:** Credentials are digitally signed; verifiers validate signatures using public keys from DID resolution.
- **Privacy by design:** Holders present only necessary credentials/claims (VP-level selective disclosure today; cryptographic selective disclosure via SD-JWT/BBS+ in the future).
- **Interoperability:** Standards-aligned DIDs/VCs/VPs reduce bespoke integrations.
- **Auditability:** Verifiers store structured evidence + optional on-chain anchoring (commitments/hashes) for tamper-evident audit trails.

4. Verification Model

Verification is the process of turning a VC/VP into a **decision** a system can safely act on. It should be a **defense-in-depth pipeline**, not a single boolean check.

○ Essential Verification Steps

1. **Parse and validate format** (JWT/JSON-LD, required fields)
2. **Resolve DIDs** (issuer/holder → DID Documents → verification methods)
3. **Verify signatures** (cryptographic proof + authorized key)
4. **Validate semantics** (types, expiration, audience/nonce)
5. **Apply policy** (is issuer trusted for this claim in this context?)
6. **Optional: verify anchoring** (SPV-like check if commitments are anchored)
7. **Produce decision** (accept/reject/step-up + structured reasons + evidence)

○ Trust is Contextual

Cryptographic validity ≠ trusted enough to act on

- **Cryptographic validity:** “Was this signed by DID X and unmodified?”
- **Authorization (domain trust):** “Is DID X allowed to assert this claim for this verifier?”
- **Assurance (risk context):** “Is this proof strong enough for the action we’re taking?”

Example: A degree VC signed by “Some DID” may be cryptographically valid, but an employer should only accept it if the issuer is an accredited university (on a trust registry/allow-list).

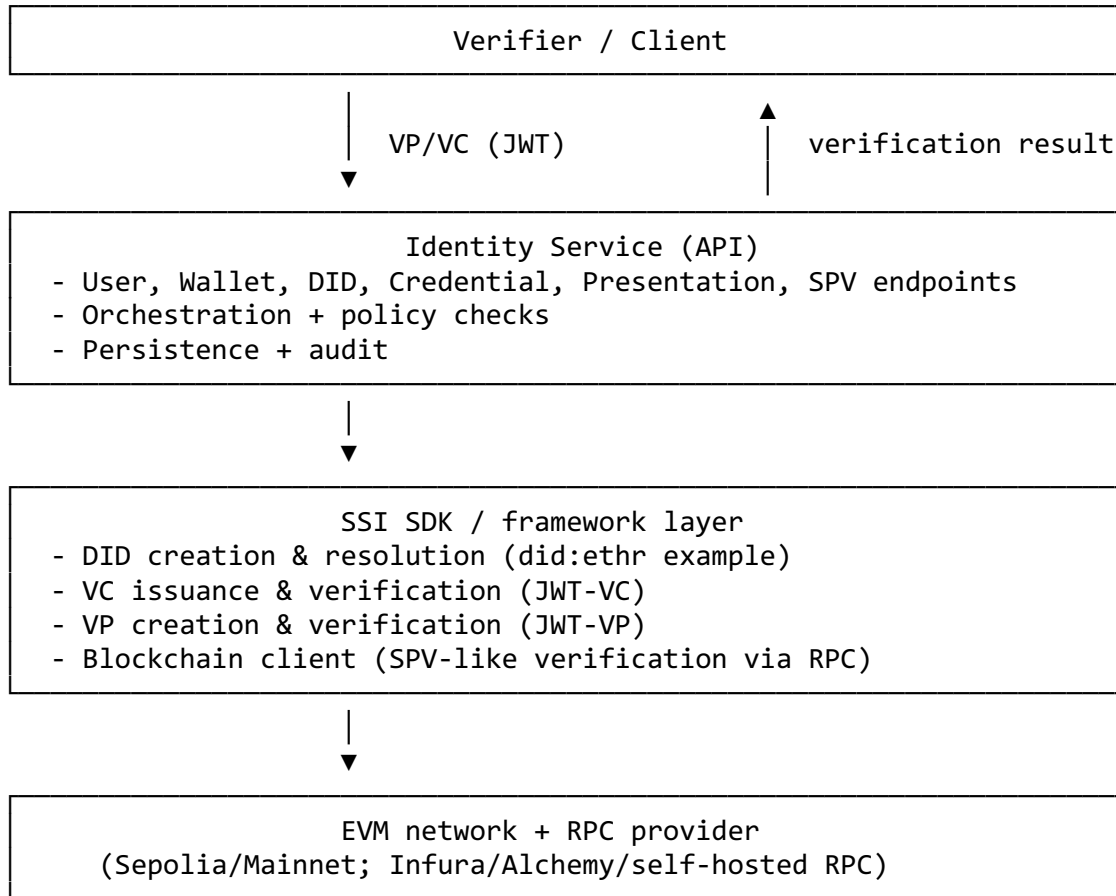
○ Guardrails for Execution

When verification gates real actions (access, transactions): - **Assurance levels:** map outcomes to risk levels - **Fail-safe defaults:** treat “cannot verify” differently from “verified false” - **Replay protection:** require nonces for VPs, validate audience - **Least privilege:** grant minimum access after success - **Audit logging:** store decisions + evidence (without logging sensitive payloads)

5. Reference Architecture (did:ethr example)

Sections 1–4 establish the “what and why.” This section focuses on “how to implement,” using a did:ethr-based reference design while staying at a high level.

○ High-level architecture



○ Workflows (implementation approach)

- *Onboard user (system boundary)*
 - Create or authenticate an internal user (for data isolation)
 - Optionally create a wallet for blockchain interactions
- *Create DID (did:ethr reference)*
 - Generate keys (secure storage recommended: KMS/HSM)
 - Create DID and DID Document

- Persist DID record and metadata; audit the operation
- *VC (JWT-VC)*
 - Issuer selects claims + credential type
 - Sign credential with issuer key
 - Compute a stable commitment (e.g., vcHash) and store credential + metadata
 - Optional: anchor the commitment on-chain (policy-driven)
- *Create VP (JWT-VP) + selective disclosure (current)*
 - Verifier requests proof (optionally includes nonce/challenge)
 - Holder selects which VCs/claims to present (data minimization)
 - Holder signs VP; verifier validates holder signature and included VCs
- *Verify VC/VP + optional SPV-like verification*
 - Decode JWT(s); validate required fields and constraints
 - Resolve issuer/holder DIDs and fetch public keys
 - Verify signatures
 - Optional: confirm on-chain anchoring evidence (SPV-like verification)
 - Return structured result with stepwise details for transparency/audit

○ Implementation checklist (capability-oriented)

To implement the above architecture, a practical system typically provides: - **DID lifecycle** - create DIDs, resolve DIDs, rotate keys (where supported by the DID method) - **Credential lifecycle** - issue credentials, store credentials securely, verify credentials - **Presentation lifecycle** - construct presentations for verifier requests, verify presentations, support challenge-binding (nonce) where applicable - **Anchoring & verification (optional)** - compute commitments/hashes for credentials/presentations - anchor commitments on-chain (policy-driven) - verify anchoring evidence (SPV-like verification) using a lightweight verifier path (e.g., RPC provider) or full inclusion proofs (advanced) - **Auditability** - correlate identity operations with timestamps, request ids, and optional chain transactions - store verification decisions and reasons (for governance/compliance)

6. Security & Operational Highlights

○ Integrity and Authenticity

- Verify signatures using public keys obtained via **DID resolution**
- Enforce algorithm compatibility (reject alg=none)
- Cache resolution results with TTL; plan for key rotation

○ Selective Disclosure

- **Today:** VP-level (holder chooses which VCs/claims to present)

- **Future:** Cryptographic claim-level (SD-JWT, BBS+) for proving specific facts without revealing others

○ Key Management

Key custody is critical: - Use **KMS/HSM** for secure storage - Define **rotation triggers** (compromise, staff changes) - **Separate keys** by role (issuer vs holder) and environment (dev/prod) - **Audit lifecycle** (creation, rotation, deactivation) without logging secrets

○ Anchoring Strategy (On-chain)

Decision matrix:

Factor	Anchor Everything	Anchor Commitments Only
Cost	High	Low (selective)
Privacy	Lower	Higher
Verifiability	Strong (public keys)	Depends on resolver trust
Scalability	Lower	Higher (batching, L2)

Recommended: Anchor only commitments/hashes (or Merkle roots) for high-value credentials; treat anchoring as **policy-driven**, not default.

7. Getting Started

○ Essential SSI Tools

Frameworks: - **Veramo** – modular JavaScript framework for DIDs/VCs/VPs - **SpruceID** – open-source libraries (DIDKit, Credible) - **Walt.id** – SSI toolkit

DID tooling (did:ethr example): - ethr-did + ethr-did-resolver - **DIF Universal Resolver** (multi-method)

Blockchain libraries: - ethers.js / wagmi / viem (EVM interaction)

RPC infrastructure: - Infura, Alchemy, or self-hosted (geth/nethermind)

○ Quick Start (Sample Project)

- **Set up test environment** (Sepolia testnet + RPC endpoint)
- **Create a DID** (use Veramo or ethr-did; generate keys → create DID → resolve)
- **Issue a test VC** (define schema, sign with issuer DID, store + compute vcHash)
- **Verify the VC** (resolve issuer DID → verify signature → check expiration)

- **Optional: anchor** (publish vcHash on-chain; implement SPV-like check)
- **Test with a VP** (holder creates VP with challenge/nonce; verify VP + embedded VC)

○ Practical Guidance

- Start with **Sepolia** for development
- Treat anchoring as a **policy decision** (cost vs auditability)
- Use **KMS/HSM** for key custody and audit all operations
- Leverage existing SDKs; don't reinvent cryptographic primitives

8. Conclusion

Self-Sovereign Identity shifts digital trust from “online verification against centralized providers” to **portable, cryptographically verifiable proofs**. By separating roles, standardizing artifacts (DIDs/VCs/VPs), and applying decision-grade verification (crypto checks + policy + optional anchoring), systems achieve: - **Integrity and interoperability** - **Privacy by design** (minimize disclosure) - **Auditable verification** outcomes

Key Takeaways:

- **Cryptography is necessary but not sufficient:** signatures prove integrity; policy determines trust.
- **Minimize disclosure:** use VPs for selective presentation; plan for SD-JWT/BBS+ for claim-level privacy.
- **Anchor selectively:** publish only commitments when audit-grade evidence is needed; treat SPV-like verification as additive assurance.
- **Engineer for safety:** map verification outcomes to actions with replay protection, least privilege, and explainable evidence.

9. Glossary (Essential Terms)

DID (Decentralized Identifier) – A globally unique identifier controlled by its subject, resolving to a DID Document with public keys and service endpoints.

DID Document – JSON-LD document containing verification methods, keys, and service endpoints for a DID.

VC (Verifiable Credential) – A tamper-evident, digitally signed credential containing claims about a subject (e.g., degree, employment status).

VP (Verifiable Presentation) – A holder-created package of one or more VCs, signed to prove possession and control.

SPV-like Verification – A lightweight method for verifying on-chain anchoring evidence (commitments/hashes) without running a full blockchain node, often via RPC-based confirmation checks.

JWT-VC / JWT-VP – Verifiable credentials/presentations encoded as JSON Web Tokens, signed by issuer/holder.

SD-JWT – Selective Disclosure JWT; enables cryptographic claim-level disclosure.

KMS/HSM – Key Management Service / Hardware Security Module; secure key storage systems.

10. Key References

W3C Standards: - **Verifiable Credentials Data Model 1.1** – <https://www.w3.org/TR/vc-data-model/> - **Decentralized Identifiers (DIDs) v1.0** – <https://www.w3.org/TR/did-core/>

DID Method (did:ethr example): - did:ethr Specification – <https://github.com/decentralized-identity/ethr-did-resolver/blob/master/doc/did-method-spec.md>

Selective Disclosure: - SD-JWT (IETF Draft) – <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-selective-disclosure-jwt>

Bitcoin SPV (Original Concept): - Bitcoin Whitepaper (Section 8) – <https://bitcoin.org/bitcoin.pdf>

Frameworks: - **Veramo** – <https://veramo.io/> - **SpruceID** – <https://www.spruceid.com/>